



**EAST TEXAS A&M**  
— UNIVERSITY —

# CSCI 380.61E WEB PROGRAMMING AND INTERFACE

# HTML DOM AND BROWSER BOM

- Component-based architecture
- Virtual DOM for faster rendering
- Declarative programming style
- Unidirectional data flow

# JSX SYNTAX

- JavaScript XML - a syntax extension for JavaScript
- Allows HTML-like code within JavaScript
- Example:

```
const element = <h1>Hello, World</h1>;
```

- Transpiled to `React.createElement('h1', null, 'Hello, World')`

# STATE AND PROPS IN REACT

- Props: Immutable data passed from parent to child components
- State: Mutable data within a component
- Example of state in a functional component (with useState hook):

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count +  
1)}>Count: {count}</button>;  
}
```

# EVENT HANDLING IN REACT

- React uses a synthetic event system

```
function MyButton() {  
  const handleClick = () => alert("Button clicked!");  
  return <button onClick={handleClick}>Click  
me</button>;  
}
```

# REACT LIFECYCLE (CLASS COMPONENTS)

- Component Lifecycle Methods:
- `componentDidMount()`
- `shouldComponentUpdate()`
- `componentWillUnmount()`

# REACT LIFECYCLE (CLASS COMPONENTS)

- Example:

```
class MyComponent extends React.Component {  
  componentDidMount() {  
    console.log('Component mounted!');  
  }  
  render() {  
    return <div>Hello React!</div>;  
  }  
}
```

# REACT HOOKS

- `useState`: For managing local state
- `useEffect`: For side effects like fetching data, timers

```
useEffect(() => {  
  fetchData();  
}, []); // Empty dependency array means this effect  
runs once, like componentDidMount
```



# HTML DOM

- The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of the HTML document as a tree of objects (nodes).
- Allows JavaScript to dynamically interact with web pages
- Provides a way to access and manipulate elements in HTML
- DOM Tree Structure:
- document is the root node
- HTML tags like `<html>`, `<body>`, `<div>`, etc. are the child nodes.

# NAVIGATING THE DOM TREE

- DOM Nodes:
- Element Nodes: Represent HTML elements (e.g., `<div>`, `<span>`)
- Text Nodes: Contain the text inside HTML tags
- Attribute Nodes: Represent the attributes of elements (e.g., `id`, `class`)

## EXAMPLE OF DOM TREE:

<html>

  <head></head>

  <body>

    <div id="content">

      <p>Hello World</p>

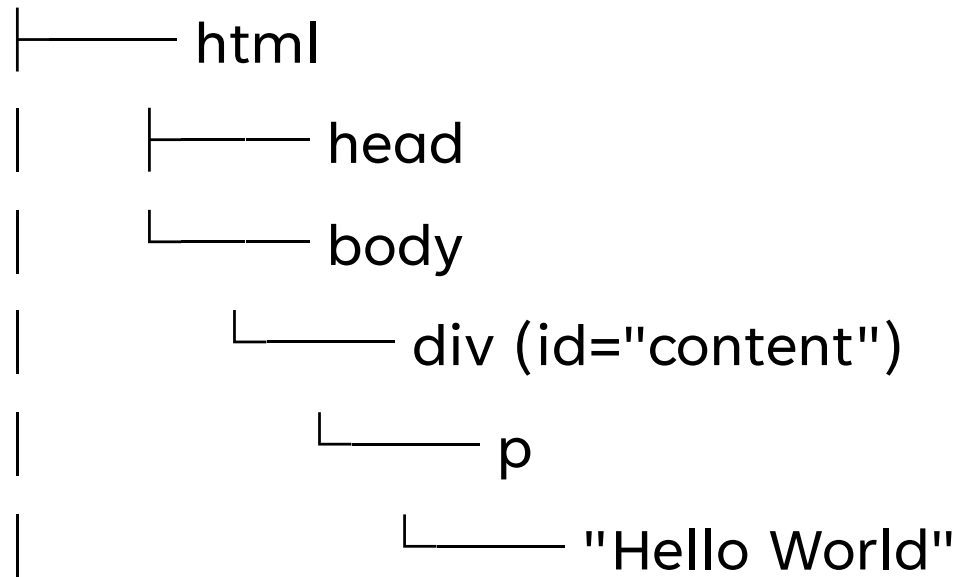
    </div>

  </body>

</html>

# VISUAL REPRESENTATION

document



# DOM MANIPULATION WITH JAVASCRIPT

- Accessing Elements:
- `document.getElementById('id')`
- `document.querySelector('.class')`
- Changing Content:

```
const element = document.getElementById('demo');  
element.innerHTML = "New content";
```

# ACCESSING DOM ELEMENTS

- `getElementById()`: Returns the element with the specified id.

```
const element = document.getElementById('content');  
console.log(element);
```

# ACCESSING DOM ELEMENTS

- `getElementsByClassName()`: Returns a list of elements with the specified class

```
const items =  
document.getElementsByClassName('list-item');  
console.log(items);
```

# ACCESSING DOM ELEMENTS

- `querySelector()`: Returns the first element matching a CSS selector.

```
const heading = document.querySelector('h1');  
console.log(heading);
```



# ACCESSING DOM ELEMENTS

- `querySelectorAll()`: Returns all elements matching a CSS selector.

```
const paragraphs = document.querySelectorAll('p');  
console.log(paragraphs);
```

# MANIPULATING DOM ELEMENTS

- Changing Inner HTML:

```
document.getElementById('content').innerHTML = 'New Content!';
```

- Changing Styles:

```
document.getElementById('content').style.backgroundColor = 'blue';
```

- Changing Attributes:

```
const link = document.querySelector('a');
```

```
link.setAttribute('href', 'https://www.example.com');
```

# CREATING AND INSERTING DOM ELEMENTS

- Creating Elements:

```
const newDiv = document.createElement('div');
```

```
newDiv.innerHTML = 'This is a new div!';
```

```
document.body.appendChild(newDiv);
```

# CREATING AND INSERTING DOM ELEMENTS

- Inserting Before/After:

```
const newElement = document.createElement('p');  
newElement.textContent = 'Inserted paragraph';  
const refElement = document.getElementById('content');  
document.body.insertBefore(newElement, refElement);
```

# REMOVING DOM ELEMENTS

- Removing an Element:

```
const element = document.getElementById('removeMe');  
element.remove();
```

- Removing a Child Node:

```
const parent = document.getElementById('parentDiv');  
const child = document.getElementById('childDiv');  
parent.removeChild(child);
```

# DOM EVENTS

- An event is an action that can be handled in JavaScript, such as a click, keypress, or hover.

- Event Listener Example:

```
const button = document.getElementById('btn');  
button.addEventListener('click', () => {  
  alert('Button was clicked!');  
});
```

- Other Event Types: click, submit, mouseover, keydown, etc.

# EVENT PROPAGATION (BUBBLING AND CAPTURING)

- **Event Bubbling:** Events propagate from the target element up to the root (default behavior).
- **Event Capturing:** Events propagate from the root down to the target element.
- **Example:**

```
document.getElementById('parentDiv').addEventListener('click', () => {  
    console.log('Parent Div Clicked!');  
}, true); // Capture phase
```

# DOM MANIPULATION EXAMPLE

- Changing styles dynamically:

```
const div = document.querySelector('div');
```

```
div.style.backgroundColor = 'blue';
```



# DOM TRAVERSING

- Accessing Parent and Sibling Elements:
- parentNode, previousSibling, nextSibling
- Example:

```
const currentElement =  
document.getElementById('current');  
const parentElement = currentElement.parentNode;  
console.log(parentElement);
```

# DOM TRAVERSING

- Example of Traversing a Node Tree:

```
const div = document.querySelector('div');
```

```
const firstChild = div.firstChild; // The first child  
element
```

```
const lastChild = div.lastChild; // The last child  
element
```

# DOM EXAMPLE - INTERACTIVE FORM

```
<form id="myForm">
```

```
  <input type="text" id="name" placeholder="Enter  
Name">
```

```
  <button type="submit">Submit</button>
```

```
</form>
```

```
<p id="greeting"></p>
```

```
<script>
```

# DOM EXAMPLE - INTERACTIVE FORM

```
document.getElementById('myForm').addEventListener('submit', (event) => {  
    event.preventDefault(); // Prevent form submission  
    const name =  
document.getElementById('name').value;  
    document.getElementById('greeting').innerHTML =  
`Hello, ${name}!`;  
});  
</script>
```

# BROWSER BOM

- The Browser Object Model (BOM) provides interaction with the browser's environment and functionality outside of the DOM.
- Key Objects in BOM:
  - window: Represents the browser window
  - navigator: Provides information about the browser
  - screen: Provides information about the screen
  - location: Provides information and methods for the URL
  - history: Allows navigation through the browser's history

# THE WINDOW OBJECT

- **Window Properties:**

- `window.innerWidth`: Window's inner width
- `window.innerHeight`: Window's inner height
- `window.location`: Current URL of the window

- Example:

```
console.log(window.innerWidth); // Get window width
```

```
console.log(window.location.href); // Get current URL
```

# THE WINDOW OBJECT

- Alert and Prompt:

```
window.alert("This is an alert!");
```

```
const userName = window.prompt("What is your  
name?");
```

```
console.log(userName);
```

# THE NAVIGATOR OBJECT

- Navigator Properties:
- `navigator.userAgent`: Returns the browser's user-agent string
- `navigator.language`: Returns the browser's language setting
- Example:

```
console.log(navigator.userAgent); // Get browser information
```

```
console.log(navigator.language); // Get language preference
```



# THE SCREEN OBJECT

- Screen Properties:
- `screen.width`: Screen width in pixels
- `screen.height`: Screen height in pixels
- `screen.availWidth`: Available screen width (excluding taskbars, etc.)
- Example:

```
console.log(screen.width); // Get screen width  
console.log(screen.height); // Get screen height
```

# THE LOCATION OBJECT

- Properties of location:
- `location.href`: Full URL of the current page
- `location.protocol`: Protocol (e.g., `http:` or `https:`)
- `location.hostname`: Domain name or IP address
- Methods:
  - `location.reload()`: Reload the page
  - `location.assign(url)`: Navigate to a new URL

```
console.log(location.href); // Get current URL
```

```
location.assign("https://www.example.com"); // Navigate to new URL
```

# THE HISTORY OBJECT

- History Methods:
- `history.back()`: Go back one step in the history
- `history.forward()`: Go forward one step in the history
- `history.go(-1)`: Go back one step (similar to `back()`)
- Example:

`history.back();` // Go back one page

`history.forward();` // Go forward one page

# USING THE WINDOW OBJECT FOR NAVIGATION

- Example of navigation using the window object:

```
// Open a new window
```

```
const newWindow =  
window.open('https://www.example.com', '_blank');
```

```
// Close the new window
```

```
newWindow.close();
```

# WORKING WITH POPUPS

- Example of opening and interacting with a popup:

```
const popup = window.open('popup.html', 'Popup Window', 'width=600,height=400');
```

```
popup.document.write('<h1>Welcome to the Popup!</h1>');
```

# THE LOCALSTORAGE AND SESSIONSTORAGE

- `localStorage`: Stores data without expiration. Data persists even after the browser is closed.

```
localStorage.setItem('username', 'John');  
const username = localStorage.getItem('username');  
console.log(username); // Outputs 'John'
```

# THE LOCALSTORAGE AND SESSIONSTORAGE

- `sessionStorage`: Stores data for the duration of the page session. Data is lost when the page is closed.

```
sessionStorage.setItem('theme', 'dark');
```

```
const theme = sessionStorage.getItem('theme');
```

```
console.log(theme); // Outputs 'dark'
```

# HTML INPUT AND FORMS

- Types of input elements: `<input type="text">`
- `<input type="radio">`
- `<input type="checkbox">`
- `<select>`, `<textarea>`



# HANDLING FORM INPUT IN JAVASCRIPT

- Getting Values:

```
const input = document.getElementById('inputId');  
console.log(input.value);
```

# FORM VALIDATION

- Basic HTML5 Validation:

```
<form>
```

```
  <input type="text" required>
```

```
  <input type="submit">
```

```
</form>
```

# FORM VALIDATION WITH JAVASCRIPT

```
const form = document.querySelector('form');  
form.onsubmit = function(event) {  
  const input =  
    document.querySelector('input[type="text"]');  
  if (input.value.trim() === '') {  
    alert('Please fill out the field!');  
    event.preventDefault();  
  }  
};
```

# JAVASCRIPT OBJECTS

- A collection of key-value pairs.

```
const person = {  
  name: 'John',  
  age: 30,  
  greet: function() {  
    console.log("Hello!");  
  }  
};
```

# OBJECTS AND ARRAYS

- Arrays as object properties:

```
const student = {  
  name: 'Alice',  
  subjects: ['Math', 'Science']  
};  
console.log(student.subjects[0]); // Math
```

# ES6 OBJECT FEATURES

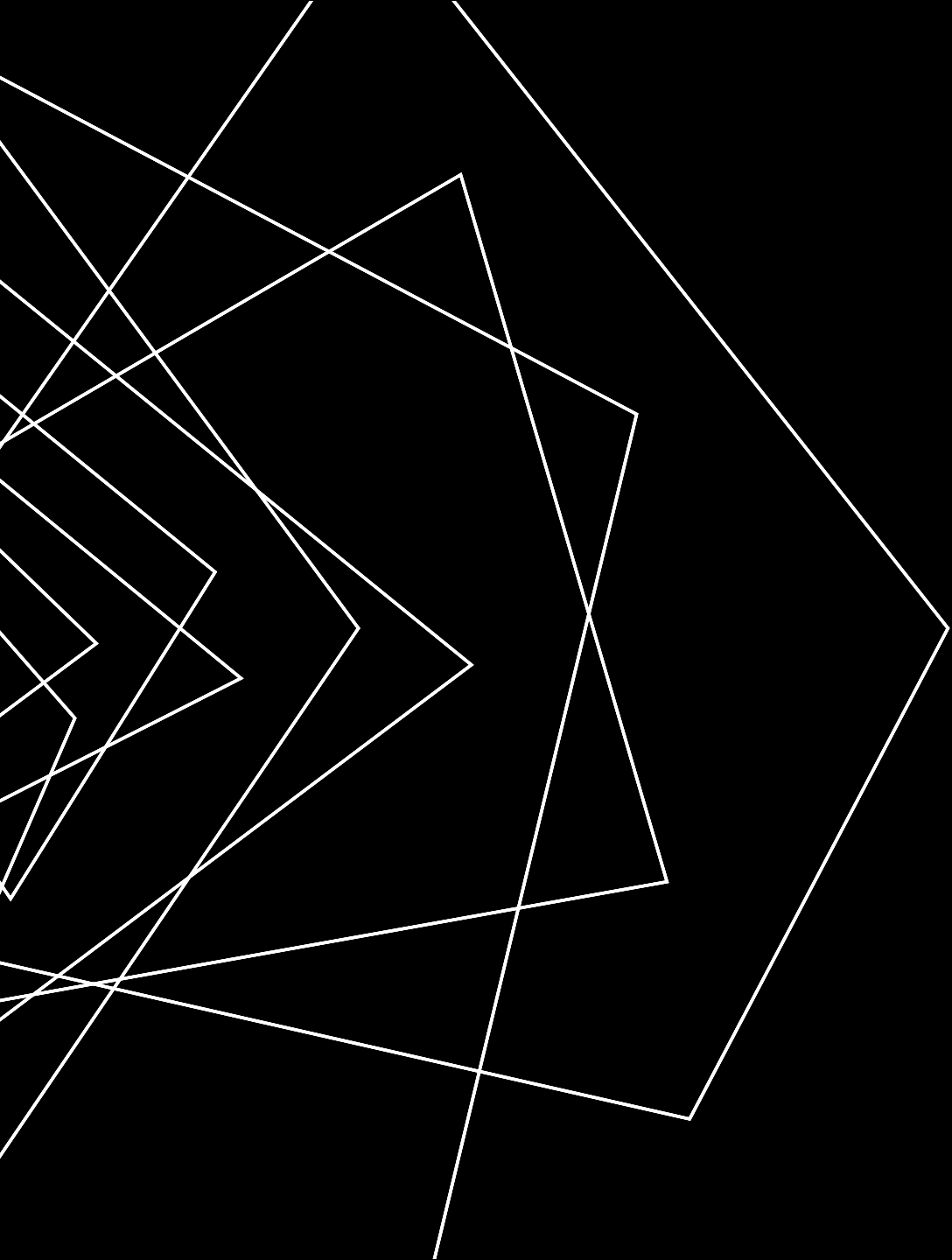
- Object Destructuring:

```
const {name, age} = person;
```

```
console.log(name); // John
```

- Object Spread Operator:

```
const newPerson = {...person, country: 'USA'};
```



# THANK YOU

Yau Ka Cheung  
Yauka.Cheung@etamu.edu  
ACB1 Room 335